CAPSTONE B · SPRINT 4 · FINAL SHOWCASE

Buildmart

Secure Retail System

A full-stack e-commerce platform for hardware and home improvement — HTML/CSS/JS front end, Node.js + Express API, SQLite database.

Group 2 · Ranjan (Group Leader), Kalyan, Shailendra, Diya, Dipesh, Sujan, Anjan

6	12	77	73
entities	API endpoints	automated tests	commits

What we will cover

1 **Problem & Overview**

Why Buildmart exists, who it serves

2 **Design & Planning**

Requirements, business rules, sprint journey

3 **Database**

6 entities, ERD, data dictionary

4 **Development**

Front end, back end, 12 API endpoints

5 **Integration**

How a click reaches the database and back

6 **CRUD & Testing**

Create, Read, Update, Delete + 77 tests

7 **Evidence**

GitHub, CI, team contributions

8 **Live Demonstration**

The working system, end to end

9 **Reflection**

Challenges, lessons, what comes next

The problem we set out to solve

The situation

Buildmart is a hardware and home improvement retailer. Its customers need to browse stock, build a basket across a visit, and trust that what they see is what is actually available.

What was wrong

- Product data lived in the page, so prices and stock could drift from reality
- The cart existed only in browser memory - a refresh lost everything
- Nothing stopped a customer ordering more than existed in stock
- No accounts, so no way to recognise a returning customer

What we built

A three-tier system where the database is the single source of truth. Every product, price, stock level and cart line is read from and written to SQLite through a REST API.

- Products load from the database, never the page
- The cart survives a refresh - it lives in SQLite
- Stock and quantity limits enforced server-side
- Secure accounts with bcrypt-hashed passwords

The front end never decides what is true. It asks the API, and the API asks the database.

Target users and project objectives

WHO USES THE SYSTEM

Retail customers

Browse the 8-product catalogue, build a cart, see GST-inclusive totals, create an account and log in

Returning customers

Log in and find the cart exactly as they left it, because it is stored server-side

Store staff (future)

The schema and API already separate stock from display, ready for an admin view

PROJECT OBJECTIVES

- Move every piece of product and cart data into a real database
- Expose it through a documented REST API with full CRUD
- Validate every input server-side - never trust the browser
- Protect accounts with industry-standard password hashing
- Prove it works with automated tests and continuous integration
- Deliver it through a genuine Git workflow: issues, branches, pull requests, review

MAIN FEATURES

Product catalogue	Shopping cart	Quantity controls
GST totals	Registration	Login / logout

Requirements analysis

FUNCTIONAL REQUIREMENTS

- View all products with price, unit and stock from the database
- Add a product to the cart with a chosen quantity
- Change the quantity of a cart line
- Remove a line, or the whole item, from the cart
- See a GST-inclusive subtotal, GST component and total
- Register an account and log in and out securely

NON-FUNCTIONAL REQUIREMENTS

- Validation enforced on the server, not only in the browser
- Passwords never stored in recoverable form
- Responsive down to 375px with no sideways scrolling
- Graceful failure when the API cannot be reached

BUSINESS RULES ENFORCED BY THE SYSTEM

#	Rule	Enforced by
1	A cart line must point to a real cart and real product	Foreign keys + 404
2	Quantity must be a whole number above zero	CHECK + 400
3	At most 20 of any one product per cart	API returns 400
4	Cannot exceed available stock	API returns 409
5	A product appears only once per cart	UNIQUE(cart_id, product_id)
6	Prices include GST; GST = total - total / 1.1	getCart() in server.js
7	Usernames and emails unique in any letter case	UNIQUE COLLATE NOCASE
8	Passwords stored only as bcrypt hashes	auth/passwords.js

The sprint journey

1

Capstone A

Foundation

Static site: home, about, shop, contact. Shared stylesheet, responsive layout.

2

Sprint 2

Cart in the browser

Shopping cart module - drawer, add to cart, quantity controls, GST totals. All client-side.

3

Sprint 3

Database and API

Node.js + Express + SQLite. Products and cart tables, REST endpoints, automated tests.

4

Sprint 4

Integration and accounts

Front end rewired to the API. Accounts with bcrypt and JWT. CI, full test suite, documentation.

The turning point

Sprint 3 built a working API and Sprint 2 built a working cart - but they were not connected. Sprint 4 was mostly integration work: making the page we already had speak to the database we already had.

Database design decisions

SQLite
Chosen over MySQL

6
Entities

3
Relationships

8
Business rules

WHY SQL, NOT NoSQL

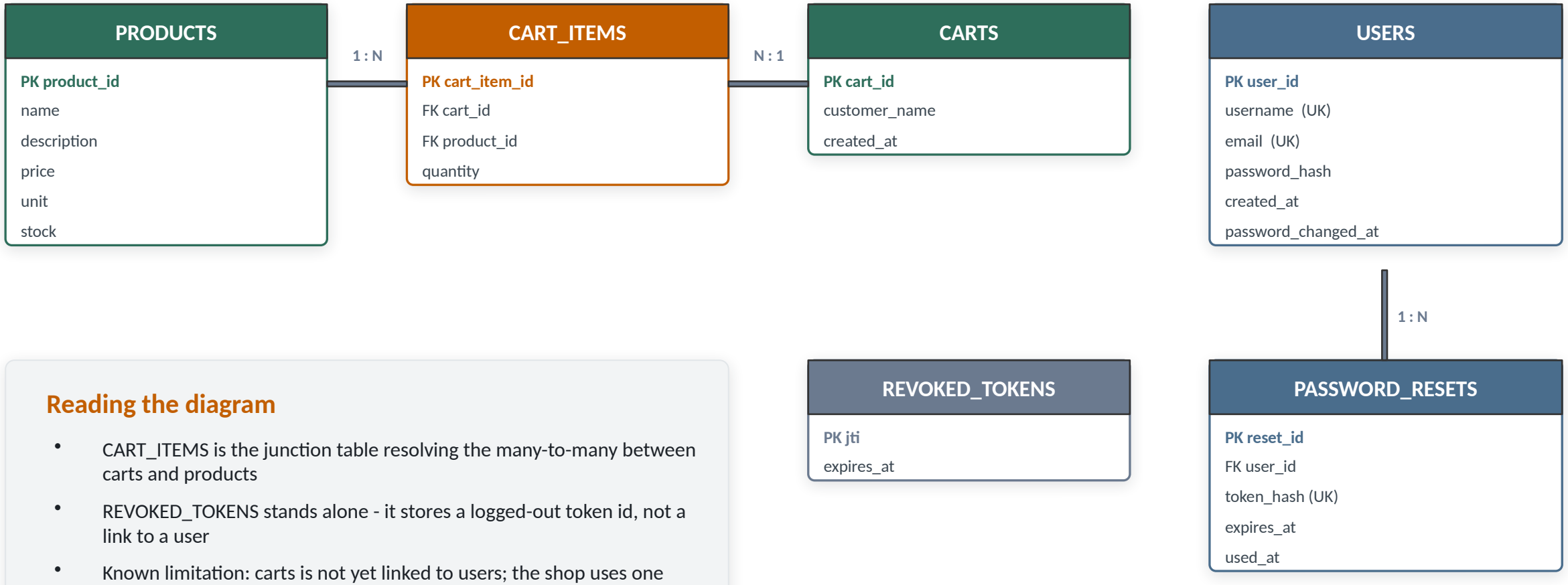
- Our data is strongly relational - a cart line is meaningless without its cart and its product
- Foreign keys let the database itself refuse an orphaned cart line
- A UNIQUE constraint on (cart_id, product_id) enforces 'one line per product' without any application code
- CHECK constraints keep quantity above zero and price at or above zero at the storage layer
- Totals and GST are a natural SQL JOIN and aggregation

WHY SQLite SPECIFICALLY

- Zero configuration - the database is a single file, so every team member runs the same thing after one npm command
- Fully relational: foreign keys, CHECK and UNIQUE constraints all supported
- better-sqlite3 is synchronous, which kept our route handlers simple and readable
- Each test file creates its own temporary database, so tests never touch buildmart.db

Trade-off we accepted: SQLite handles one writer at a time. For a single-store prototype that is fine; a multi-store deployment would move to MySQL or PostgreSQL, and the schema would port directly.

Entity relationship diagram



Reading the diagram

- CART_ITEMS is the junction table resolving the many-to-many between carts and products
- REVOKED_TOKENS stands alone - it stores a logged-out token id, not a link to a user
- Known limitation: carts is not yet linked to users; the shop uses one shared cart

Data dictionary and constraints

Table	Column	Type	Constraint	Purpose
products	product_id	INTEGER	PRIMARY KEY	Unique product number
	price	REAL	NOT NULL, CHECK >= 0	AUD, GST inclusive
	stock	INTEGER	NOT NULL, CHECK >= 0	Units available
carts	cart_id	INTEGER	PRIMARY KEY	Unique cart number
	created_at	TEXT	NOT NULL, DEFAULT now	When the cart was created
cart_items	cart_item_id	INTEGER	PRIMARY KEY	Unique line number
	cart_id	INTEGER	NOT NULL, FK -> carts	Owning cart
	product_id	INTEGER	NOT NULL, FK -> products	Product on this line
	quantity	INTEGER	NOT NULL, CHECK > 0	Units ordered
	(cart_id, product_id)	-	UNIQUE	One line per product per cart
users	username / email	TEXT	UNIQUE COLLATE NOCASE	Case-insensitive identity
	password_hash	TEXT	NOT NULL	bcrypt hash, cost 12

Every constraint above is created by backend/init-db.js and backend/auth/schema.js, so a fresh clone plus one npm command reproduces this exact schema. Nothing is configured by hand.

Front-end development

BUILT WITH

Plain HTML5, CSS3 and vanilla JavaScript - no framework. Six pages sharing one stylesheet.

shop.html	Catalogue + cart drawer
register.html	Validation + strength meter
login.html	Login / logout session
index.html	Home and hero
about.html	Project and team
contact.html	Enquiry form

WHAT THE FRONT END DOES - AND DOES NOT DO

Does

- Render what the API returns
- Validate early for fast feedback
- Show loading and error states
- Keep the token in sessionStorage
- Re-draw the cart from each response

Does not

- Hold the cart in memory
- Calculate the total itself
- Decide what is in stock
- Hard-code any product or price
- Trust its own validation alone

REGISTRATION FORM - REAL-TIME VALIDATION

Username	3-30 characters, letters, numbers and underscore
Email	Format checked as you type, uniqueness checked by the API
Password	Live checklist: length, uppercase, lowercase, number, symbol
Confirm	Must match, checked on every keystroke
Strength meter	Updates live as the rules are satisfied

Client-side validation is a courtesy to the user. The server re-checks everything, because a browser can be bypassed.

Back-end development

THE STACK

Node.js 20	JavaScript runtime
Express 4.21	HTTP routing and middleware
better-sqlite3 12	Synchronous SQLite driver
bcryptjs 3	Password hashing, cost factor 12
jsonwebtoken 9	HS256 tokens, 1-hour expiry
node:test	Built-in test runner, no extra dependency

Security measures in the code

- JWT secret read from the environment, never committed
- Server refuses to start in production without JWT_SECRET
- Login returns one message for a wrong password and an unknown email - no user enumeration
- Static file allow-list: only root .html plus /css and /js are served

WHAT THE BACK END IS RESPONSIBLE FOR

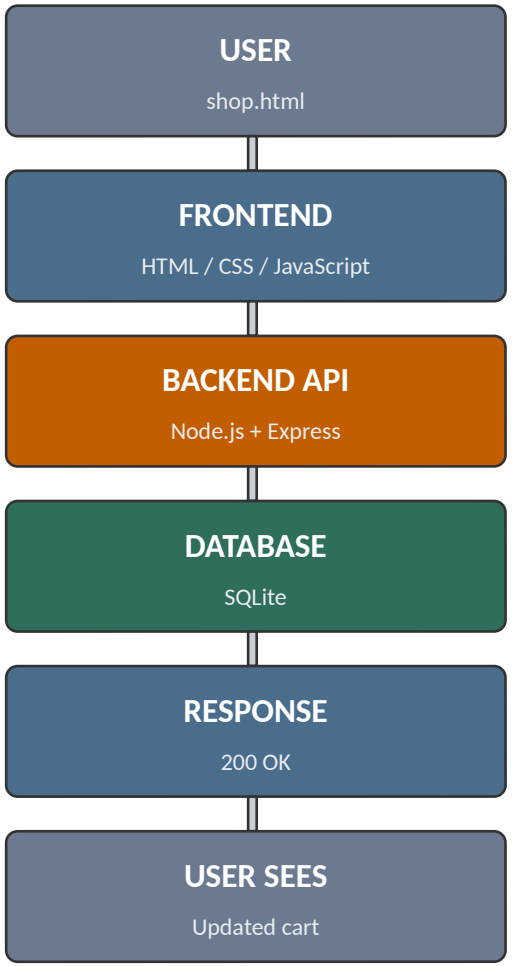
1. **Validate**
   Every id, every quantity, every field - before touching the database
2. **Authorise**
   Check the bearer token and that it has not been revoked
3. **Enforce rules**
   Stock levels, the 20-item cap, one line per product
4. **Query**
   Prepared SQL statements - never string concatenation
5. **Calculate**
   Line subtotals, cart total and the GST component
6. **Respond**
   Correct HTTP status and a consistent JSON shape

API endpoints

GET	`/api/products`	List all 8 products with price, unit and stock	Read
GET	`/api/carts/:id/items`	Cart contents with line subtotals, GST and total	Read
POST	`/api/carts/:id/items`	Add a product to the cart, or raise its quantity	Create
PATCH	`/api/carts/:id/items/:productId`	Set the quantity of an existing cart line	Update
DELETE	`/api/carts/:id/items/:productId`	Remove a line from the cart	Delete
POST	`/auth/register`	Create an account; stores a bcrypt hash	Create
POST	`/auth/login`	Verify credentials and issue a JWT	Read
GET	`/auth/verify`	Validate the bearer token, return the user	Read
POST	`/auth/logout`	Revoke the token by its jti	Update
POST	`/auth/forgot-password`	Issue a single-use, 30-minute reset token	Create
POST	`/auth/reset-password`	Consume the token and set a new password	Update

11 application endpoints plus a health check at GET / · Every one returns JSON · Every one is covered by the automated test suite

How information flows through the system



Clicks + on the Claw Hammer line in the cart drawer

shop.html sends PATCH /api/carts/1/items/2 with {"quantity": 5}

Express validates the id and quantity, checks the 20-item cap and the stock level

UPDATE cart_items SET quantity = 5 WHERE cart_id = 1 AND product_id = 2

JOIN recalculates the line subtotal, cart total and GST, returned as JSON

The drawer re-draws: quantity 5, line total $124.75, GST included $11.34

One click. Six stages. The browser never calculates the total and never decides what is in stock - it asks, and it renders the answer.

CRUD functionality — Create and Read

POST **Create**

`POST /api/carts/1/items`

Adds a product to the cart. If the product is already in the cart, the quantity is raised instead of creating a duplicate row.

VALIDATION PERFORMED

- product_id must be a positive integer
- quantity must be a positive integer
- The product must exist, or 404
- Combined quantity must stay at or under 20, or 400
- Combined quantity must not exceed stock, or 409

VERIFIED RESULT

```
201 Created
{"cart_id":1,"items":[{"product_id":2,
 "name":"Claw Hammer 450g","quantity":2,
 "subtotal":49.9}],"gst_included":4.54,"total":49.9}
```

GET **Read**

`GET /api/products · GET /api/carts/1/items`

Lists the catalogue, and returns the cart with a SQL JOIN that attaches each product's name and price to its cart line.

VALIDATION PERFORMED

- Cart id must be a positive integer, or 400
- The cart must exist, or 404
- Subtotals computed in SQL, not in the browser
- GST derived as total - total / 1.1
- Rows ordered by cart_item_id for a stable display

VERIFIED RESULT

```
200 OK
[{"product_id":1,"name":"Bolt",
 "description":"Zinc plated, M10","price":12,
 "unit":"each","stock":50}, ... 8 products]
```

CRUD functionality — Update and Delete

PATCH

Update

`PATCH /api/carts/1/items/2`

Sets the quantity of an existing cart line to an exact value, then recalculates the line subtotal, cart total and GST.

VALIDATION PERFORMED

- product_id must be a positive integer
- quantity must be a positive integer
- The cart line must already exist, or 404
- Quantity must stay at or under 20, or 400
- Quantity must not exceed stock, or 409

VERIFIED RESULT

```
200 OK
{"items":[{"product_id":2,"quantity":5,
 "subtotal":124.75}],
 "gst_included":11.34,"total":124.75}
```

DELETE

Delete

`DELETE /api/carts/1/items/2`

Removes a line from the cart entirely, whatever its quantity, and returns the recalculated cart so the page can re-draw.

VALIDATION PERFORMED

- Cart id and product_id must be valid integers
- The cart must exist, or 404
- The cart line must exist, or 404
- Deletion is confirmed by result.changes
- The updated cart is returned, so totals stay in step

VERIFIED RESULT

```
200 OK
{"cart_id":1,"items":[],
 "gst_included":0,"total":0}
```

Testing evidence

77
automated tests

77
passing

0
failing

25
manual integration tests

AUTOMATED SUITE — npm test

Test file	Tests	Covers
auth.test.js	45	Hashing, register, login, verify, logout, reset
cart.test.js	20	Full CRUD, stock, 20-item cap, totals, GST
static.test.js	7	Page serving and path-traversal defence
seed.test.js	5	The 8 seeded products and cart 1
Total	**77**	**All passing**

MANUAL INTEGRATION TESTS — TESTING.md

ID	Category	Test	Result
T01	Read	Products load from the database	Pass
T02	Read	Cart survives a page refresh	Pass
T03-04	Create	Add item; add the same item again	Pass
T05-06	Update	Increase and decrease quantity	Pass
T07-08	Delete	Remove item; decrease to zero	Pass
T09	Calculation	Totals and GST: $128.00 / $11.64	Pass
T10-11	Validation	Stock limit; 20-item cap	Pass
T19-22	Accounts	Register, login, refresh, logout	Pass

Every test was run against the real system on 1 November 2026. Screenshots for the key tests are committed in docs/screenshots/, named by test ID.

Validation and error handling

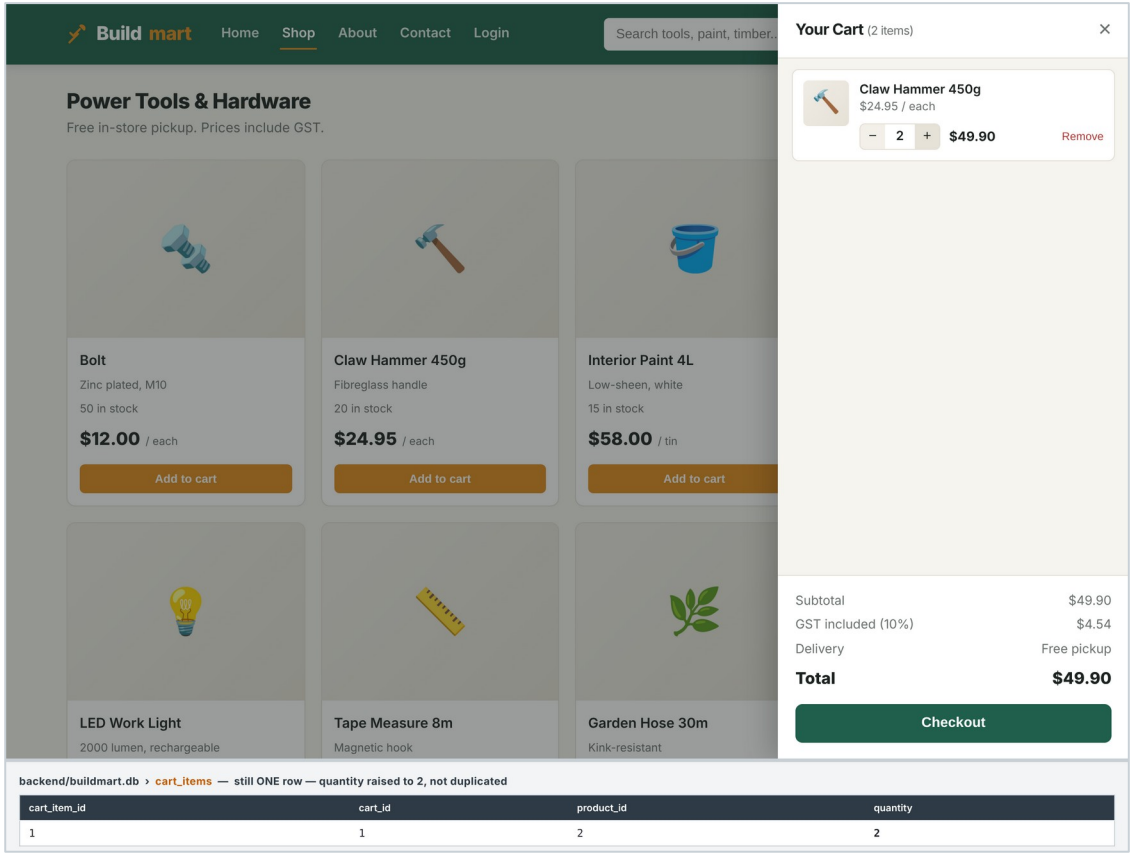
Scenario	What we sent	Status	Message returned	Database
Blank field, missing field, or wrong data type	quantity omitted, {} sent, or quantity: "abc" / -2	400	product_id must be a positive integer and quantity must be a positive integer	Unchanged
Incorrect product id	product_id: 999	404	Product not found	Unchanged
Incorrect cart id	cart 999	404	Cart not found	Unchanged
Incorrect cart item	PATCH a product not in the cart	404	Cart item not found	Unchanged
Duplicate record	same product added twice	201	No duplicate row - quantity raised	1 row, qty 2
Over stock limit	4 x Safety Glasses (stock 3)	409	Requested quantity exceeds available stock	Stays at 3
Over the 20 cap	21 x Bolt (stock 50)	400	Maximum quantity per item is 20	Stays at 20
Wrong password	valid email, bad password	401	Invalid email or password	No session
Unknown email	email not registered	401	Invalid email or password	No session
Server unreachable	API stopped mid-session	-	Could not reach the server	Unchanged

Two deliberate choices. Row 1: blank, missing and wrong-type inputs share one message because the handler validates both fields in a single condition. Rows 7 and 8: a wrong password and an unknown email answer identically, so an attacker cannot discover which emails have accounts.

CRUD evidence — Create and Update

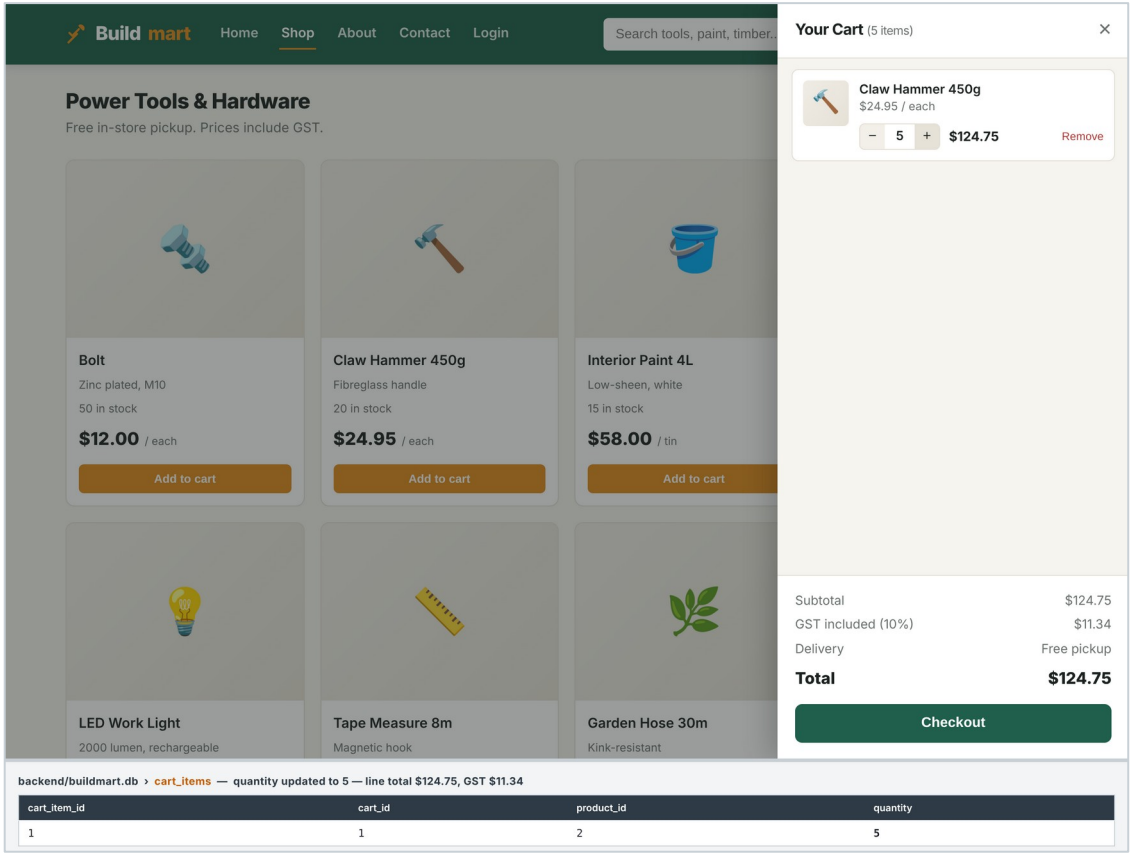
CREATE — the same product added twice

One row, quantity raised to 2 — UNIQUE(cart_id, product_id) prevents a duplicate



UPDATE — quantity set to 5

Line total $124.75, GST included $11.34, and quantity 5 in the table

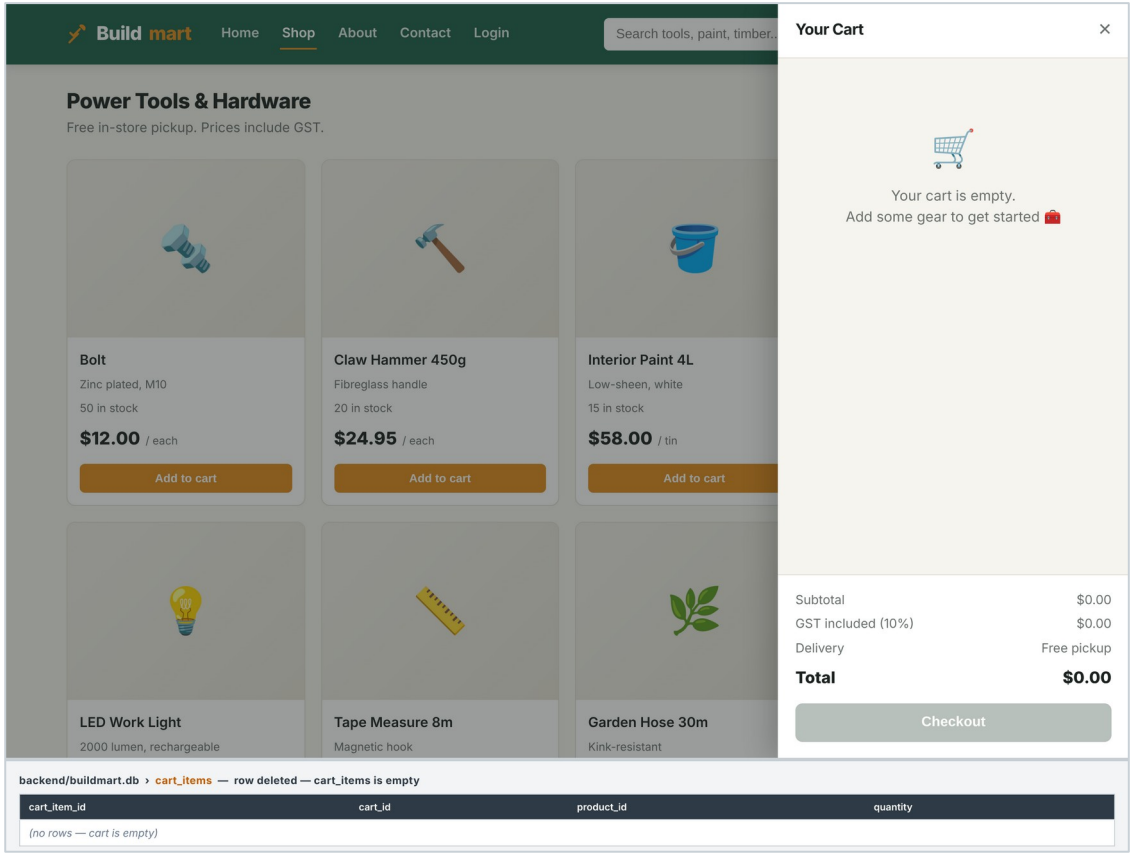


Captured from the running system: the page on top, the cart_items table underneath. The numbers on the page and in the database match.

CRUD evidence — Delete and stock validation

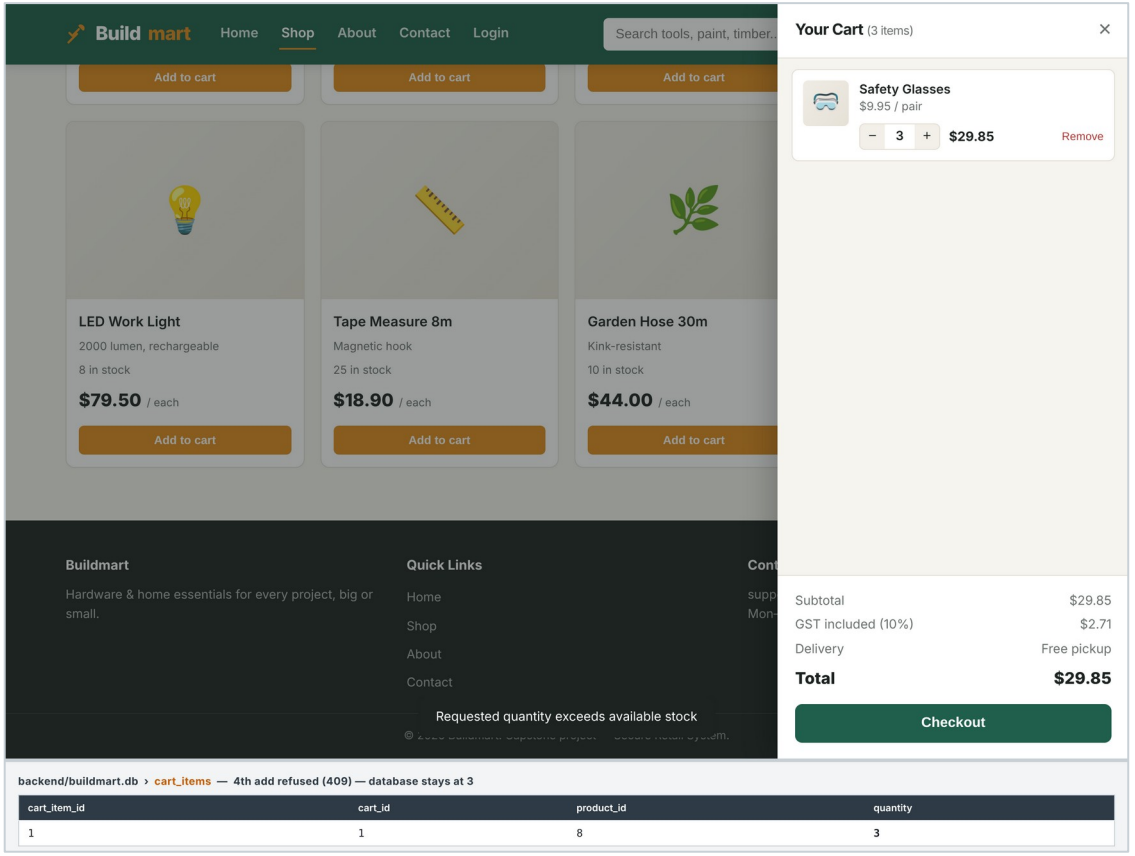
DELETE — the line removed

Drawer empty and cart_items has no rows — the row is gone from the database



VALIDATION — stock limit enforced

Safety Glasses stock is 3; the 4th add is refused with 409 and the table stays at 3



The stock check happens on the server. The browser cannot talk its way past it, because the browser is not the thing deciding.

GitHub evidence

73	19	7	2
commits on main	merged pull requests	contributing members	CI workflows

THE WORKFLOW WE FOLLOWED

1	**Issue**	Every feature begins as a GitHub issue, labelled and assigned to its owner
2	**Branch**	The owner works on a feature branch cut from main
3	**Pull request**	Work is opened as a PR, never pushed straight to main
4	**Review**	Another member reviews before the merge
5	**CI**	Tests and HTML validation must pass before merging

github.com/CapstoneAB/capstone_A_B

CONTINUOUS INTEGRATION

`backend-tests.yml`

Runs all 77 tests on Node 20 for every push and pull request to main

`frontend-check.yml`

Validates every HTML page against .htmlvalidate.json

The repository also contains

Source code (front end + back end) · Schema scripts · TESTING.md · Database design and ERD · Data dictionary · Team contribution record · Test screenshots

Team contributions

Member	Responsibility	Work delivered	Evidence	Presents
Ranjan	Group leader, coordination	Project coordination and site integration	3 commits	Introduction, requirements, conclusion
Kalyan	Data and integration	Week 2 data analysis, GST in the cart API, Sprint 4 integration	PR #46, #49, #53	Database, ERD, integration, demo
Shailendra	Add to cart and backend	Contact form, add-to-cart, cart backend and REST API	PR #2, #14, #47, #48	Backend
Diya	Cart interface and login	Cart drawer, login form, HTML checks in CI	PR #15, #51	Front end
Dipesh	Quantity update	Quantity update feature, the 20-item cap	PR #4, #21	CRUD, future improvements
Sujan	Pricing and CI	GST-inclusive totals, backend test suite, GitHub Actions	PR #18, #50	Sprint journey, GitHub, team
Anjan	Testing and accounts	Edge-case tests, registration page, auth API, password hashing	PR #19, #52	Testing, reflection

Commits on main before the Sprint 4 pull request: Kalyan 24 · Sujan 11 · Anjan 11 · Diya 7 · Shailendra 5 · Ranjan 3 · Dipesh 3 · Total 64

Counts combine the different names each member has committed under. They record participation, not the size or value of the work — a single integration commit can represent a week of effort.

Live demonstration

Everything shown is a test that has already passed. Nothing untested is demonstrated.

1	READ	**Open the shop**	8 products load from SQLite - price, unit and stock all from the database
2	CREATE	**Add Claw Hammer**	Toast appears, badge shows 1, a new row appears in cart_items
3	READ	**Refresh the page**	The cart is still there - proof it lives in the database, not the browser
4	UPDATE	**Click + to quantity 5**	Line total becomes $124.75, GST $11.34, and quantity changes in the table
5	DELETE	**Click Remove**	The line disappears from the drawer and the row is gone from cart_items
6	VALIDATE	**Add 4 Safety Glasses**	Stock is 3 - the fourth is refused with a 409 and the database stays at 3
7	ACCOUNTS	**Register and log in**	The users table stores a bcrypt hash beginning $2b$12$ - never the password

Reflection and future improvements

THE HARDEST TECHNICAL PROBLEM

Connecting the front end to the back end. Our fetch requests were blocked by CORS errors, and the cart total did not refresh after an update or a delete.

How we solved it

- Express now serves the website pages itself, so the page and the API share one origin and CORS never applies
- Every cart change re-draws the drawer from the JSON the API returns, instead of from local state
- Cart changes are sent one at a time, so rapid clicks cannot arrive out of order

LESSONS LEARNED

- Build the integration early - two working halves are not a working system
- Validate on the server; the browser can always be bypassed
- Automated tests caught regressions a manual pass would have missed
- Small pull requests reviewed by a teammate were faster than large ones

KNOWN LIMITATIONS WE ARE OWNING

Carts are not linked to users

The shop uses one shared cart, so a cart is saved but does not belong to an account

No orders table

Checkout clears the cart and reports a demo checkout; no order is recorded

Stock never decreases

Stock is checked on add, but nothing decrements it, because orders do not exist

WITH MORE TIME — IN PRIORITY ORDER

1. Link carts to user accounts, so each customer has their own
2. Add orders and order_items; decrement stock inside a transaction
3. Build the forgot-password page - the API already exists
4. Harden security: rate-limit login, move the token to an httpOnly cookie
5. Extend the test suite and add structured server-side logging

Conclusion

Buildmart began Capstone B as four static pages. It ends as a working three-tier system where the database is the single source of truth.

Database
6 entities, 3 relationships, 8 business rules enforced in SQL

Back end
Express API, 11 endpoints, prepared statements throughout

Front end
6 pages that render what the API returns and nothing more

Integration
Every product, price, cart line and total flows end to end

CRUD
Create, Read, Update and Delete, all four verified live

Security
bcrypt cost 12, JWT with revocation, no user enumeration

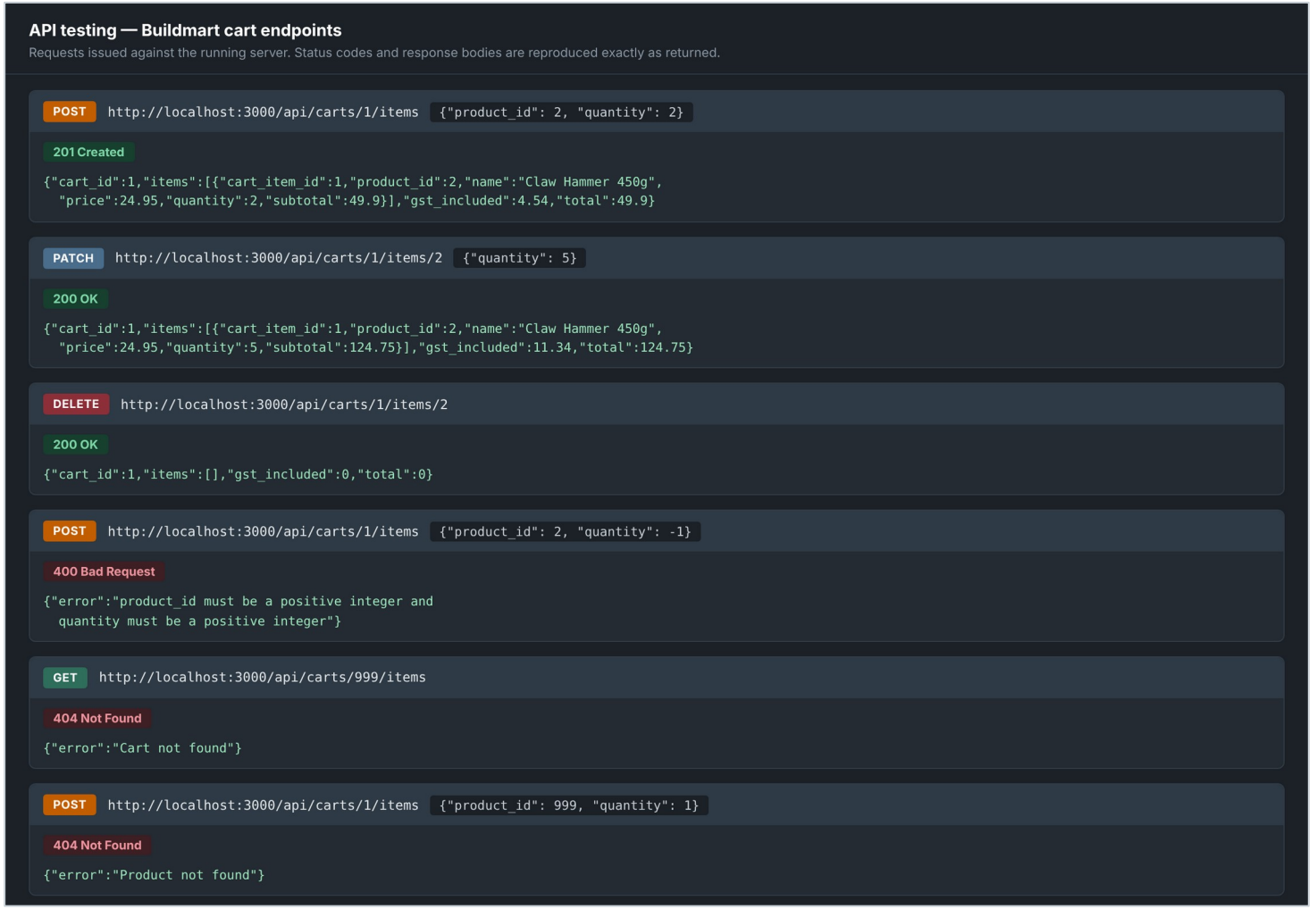
Testing
77 automated tests and 25 manual integration tests, all passing

Collaboration
73 commits, 19 reviewed pull requests, CI on every one

Thank you — we are ready for your questions

Group 2 · Ranjan · Kalyan · Shailendra · Diya · Dipesh · Sujan · Anjan · github.com/CapstoneAB/capstone_A_B

Backup — API testing evidence



Hold this slide for Q&A

Not part of the 15 minutes. Use it if a marker asks to see the API tested directly rather than through the page.

WHAT IT SHOWS

- Create returns 201 with the new cart line
- Update returns 200 with quantity 5 and total $124.75
- Delete returns 200 with an empty cart
- A negative quantity is refused with 400
- An unknown cart and an unknown product both return 404

Why it matters

The page and the API were tested separately. A fault in either one shows up on its own rather than hiding behind the other.